

2026年2月19日 星期四 22:36

In this section you will:

- review how multiplication can be implemented using shift-and-add,
- translate a structured control-flow algorithm into EEP1 assembly,
- extend your solution to support 32-bit partial multiplication.

The EEP1 ISA does not include a hardware multiply instruction, so multiplication must be implemented in software using combinations of shift and add instructions, with conditional jumps providing control flow.

```

0 // implement sum := op1 * op2 (LS 16 bits of)
1 // assume int = 16 bits (16-bit version of C)
2 unsigned int op1, op2, op2_shifted, sum; // variables
3 sum = 0; // initialise
4 op2_shifted = op2; // initialise
5 while (op1 != 0) {
6     if (op1 & 1) { // & = bitwise AND
7         sum = sum + op2_shifted;
8     }
9     op2_shifted = op2_shifted << 1; // left shift by 1
10    op1 = op1 >> 1; // right shift by 1
11 }

```

Figure 1: Shift-and-add multiplication using a while loop

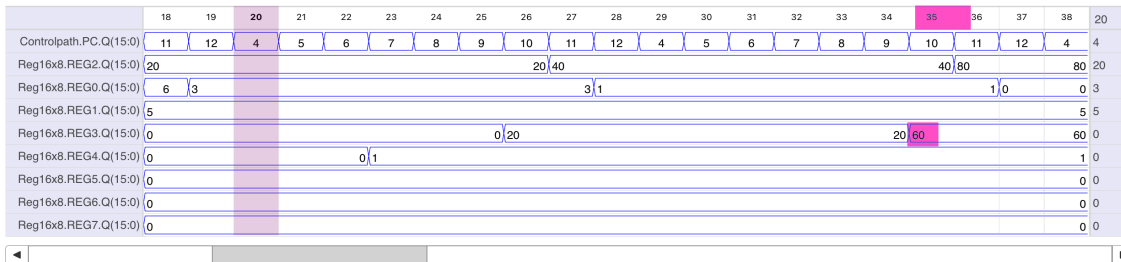
Task 3. Manually trace the code in Figure 1 with $op1 = 12$ and $op2 = 5$ to verify that it produces the result 60. Compare its behaviour with the shift-and-add multiply method explained in Lecture 3 to confirm that the algorithm is correct.

Task 4. Using the method from Lecture 4 for compiling if-then and while constructs into jumps, and the EEP1 instructions for addition, shifting, and logical AND, translate the program in Figure 1 into EEP1 assembly. Use registers R0, R1, R2, R3 to hold the four variables op1, op2, op2_shifted, sum respectively. Write your assembly program into a mul1.txt file and use eepassembler to generate a mul1.ram machine-code file. Run it on your own (now fully working) EEP1 design in Issie and check that it multiplies correctly.

variable when doing if-condition check.

```
MOV R2, R1
MOV R3, #0
```

Test for $op1 = 12$ and $op2 = 5$, we can see:



```
//Initialize:
MOV R0, #12
MOV R1, #5
MOV R2, R1    //OP2_shifted
MOV R3, #0

CMP R0, #0    //while loop
JEQ 8
```

Diagram illustrating the ALUOp field in the MIPS instruction format and its corresponding ALUOp codes.

The instruction format shows the ALUOp field (bits 26-31) and the ALUOp codes (bits 0-15).

ALUOp codes table:

ALUOp	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ADD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ADDI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SUB	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SUBI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
AND	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ANDI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
OR	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ORI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
XOR	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
XORI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLT	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLTI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLTU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLTIU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLL	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SLLI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 2: EEP1 Jump Instructions

instructions

[illegible]

Figure 2: EEP1 Jump Instructions

```

MOV R4, #1
AND R4, R4, R0 //if condition
JEQ 2
ADD R3, R3, R2
LSL R2, R2, #1
LSR R0, R0, #1
JMP -8

```

Task 5. Extend your implementation so that op2_shifted and sum are 32-bit quantities, each stored across two registers. Replace the 16-bit add and shift operations by corresponding 32-bit operations using pairs of registers.

```

//Initialize:
MOV R0, #op1
MOV R1, #op2
LSL R0, R0, #6
LSL R1, R1, #2
MOV R4, #0 //low 16 of answer
MOV R5, #0 //high 16 of answer
MOV R2, R1 //op2_shifted low 16
MOV R3, #0 //op2_shifted high 16

```

```

CMP R0, #0 //while loop
JEQ 12

```

```

MOV R6, #1 //op1_low 16 if condition
AND R6, R6, R0
JEQ 3
ADD R4, R4, R2
ADC R5, R5, R3

```

```

LSL R3, R3, #1 //left_shift of op1
LSL R2, R2, #1
JCC 2
ADD R3, #1

```

```

LSR R0, R0, #1
JMP -12

```

We test it using 2 numbers: 7232 and 424:

```

mul32.txt
//Initialize:
MOV R0, #113
MOV R1, #186
LSL R0, R0, #6 //7232
LSL R1, R1, #2 //424
MOV R4, #0 //low 16 of answer
MOV R5, #0 //high 16 of answer
MOV R2, R1 //op2_shifted low 16
MOV R3, #0 //op2_shifted high 16

CMP R0, #0 //while loop
JEQ 12

MOV R6, #1 //op1_low 16 if condition
AND R6, R6, R0
JEQ 3
ADD R4, R4, R2
ADC R5, R5, R3

LSL R3, R3, #1 //left_shift of op1
LSL R2, R2, #1
JCC 2
ADD R3, #1

LSR R0, R0, #1
JMP -12

//Result = 386368
//So expected R5 = 0x802E, R4 = 0xCA08

```

	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	126
Controlpath.PC.Q(15:0)	x0009	x000A	x000B	x000C	x000D	x000E	x000F	x0010	x0011	x0013	x0014	x0008	x0009	x000A	x000B	x000C	x000D	x000E	x000F	x0010	x0011	x0009
Reg16x8.REG02.Q(15:0)	x4000	x4000 x8000										x8000 x0000										x4000
Reg16x8.REG00.Q(15:0)	x0003	x0003 x0001										x0001 x0003										x0003
Reg16x8.REG01.Q(15:0)	x01A8											x01A8 x01A8										x01A8
Reg16x8.REG03.Q(15:0)	x000D	x000D x001A										x001A x0034										x000D
Reg16x8.REG04.Q(15:0)	x0A00	x0A00 x4A00										x4A00 xCA00										x0A00
Reg16x8.REG05.Q(15:0)	x0007	x0007 x0014										x0014 x002E										x0007
Reg16x8.REG06.Q(15:0)	x0001											x0001 x0001										x0001
Reg16x8.REG07.Q(15:0)	x0000											x0000 x0000										x0000

We can see that the result is true, and it takes 144 cycles to complete.